

Pareton — Engineering Architecture v0

Campaign pipeline: [profile](#) → [submission](#) → [provenance & build gate](#)

Scope: Stage 0 only. Correctness, perf, and cross-env stages are out of scope for v0.

1. Overview

Pareton runs optimization campaigns. A campaign takes a customer's inference profile, pins an immutable baseline (vLLM commit + environment), and accepts code patches from miners. Every submission is anchored on-chain as a content hash bound to the miner's hotkey, then passes a provenance & build gate before any benchmarking. This document specifies everything up to and including that gate.

Core invariant: every artifact a score or dispute could depend on is content-addressed and frozen for the life of a campaign. Every submission is identified by its patch hash, which threads through all downstream stages.

2. Domain objects

2.1 Profile

The customer-facing specification (see: Inference Optimization Profile doc). Stored as structured data, not just a document. Key fields: target model + quantization, serving stack + launch flags, workload shape (length distributions, prefix ratio, concurrency, serving mode), hardware SKUs, SLA thresholds, priority metric, success threshold, hard constraints.

A profile is mutable customer data. It is never referenced directly by scoring — it is compiled into a campaign manifest.

2.2 Campaign

The unit of competition. One campaign = one pinned baseline + one workload + one reward context + one time window. A profile may spawn many campaigns over time (baseline re-pins, vLLM upgrades). Scores, leaderboards, and rewards are scoped to a single campaign and never compared across campaigns.

Campaign manifest — the pin set. All fields resolved to content addresses before opening:

```
CampaignManifest {
  campaign_id:      uuid
  profile_id:       uuid           // provenance link only
  baseline_repo:    url            // vLLM upstream or customer fork
  baseline_commit:  git SHA        // exact, frozen
  base_image_digest: sha256        // container: CUDA, driver, python, deps
  gpu_skus:         [string]       // e.g. ["H200-SXM-141GB"]
  workload_trace:   sha256 + url   // request trace file, content-addressed
  sla:              { p99_ttft_ms, p99_itl_ms, quality_floor_spec }
  scoring_config:   sha256 + url   // versioned scoring parameters
  allowed_paths:    [glob]         // e.g. ["vllm/**"]
  denied_paths:     [glob]         // harness, tests, CI, scoring
  window:           { opens_at, closes_at }
  manifest_hash:    sha256 over all of the above
  customer_signoff: { approved_manifest_hash, approver, timestamp }
  status:           draft | open | closed
}
```

Pinning authority: Pareton resolves customer inputs into exact SHAs/digests and validates internal consistency (image matches baseline build requirements, trace format matches harness). The customer approves the final manifest_hash before the campaign opens — this sign-off is stored and is the defense against later baseline disputes. If the customer expresses no preference for a field, Pareton pins latest-at-open. Once status = open, the manifest is immutable; any change means a new campaign.

2.3 Submission

```
Submission {
  submission_id:    = patch_hash (sha256 of canonical diff bytes)
  campaign_id:      uuid
  hotkey:           ss58                // from extrinsic signer, never self-reported
  baseline_commit:  git SHA              // must equal manifest
  retrieval_url:    url                  // miner-hosted; IPFS CID preferred
  committed_at:    block number / timestamp
  state:           see state machine
  gate_results:     { identity, integrity, base_apply, surface, build }
  build_log_ref:    object-store key
}
```

The patch is a unified git diff against baseline_commit. Canonicalize before hashing (byte-exact file as submitted; no re-encoding). The hash is computed by the miner, committed on-chain, and independently recomputed by the validator — mismatch = reject.

3. On-chain commitment (Bittensor)

- Miners publish { baseline_commit, patch_hash, retrieval_url } via the standard commitments mechanism (set_commitment). No custom pallet required for v0.
- The hotkey is taken from the extrinsic signer. Never trust a hotkey embedded in the payload.
- Dedupe on patch_hash, first-seen wins: an identical hash committed later by a different hotkey is rejected as a copy. This is the copy-attack defense.
- Commit-reveal is a known future upgrade if content front-running (mempool sniping of the patch itself) becomes a problem; not needed for v0 because the patch content is only at retrieval_url, not on-chain.

4. Provenance & build gate (Stage 0)

Five checks, run in order, fail-fast. Each writes a structured result to gate_results. A submission that passes all five is a valid, buildable, in-bounds candidate for later stages.

- **a. Identity** — the signing hotkey is a registered miner on the subnet and eligible for this campaign (campaign open, within window). Reject otherwise.
- **b. Integrity** — fetch the diff from retrieval_url (bounded size, e.g. ≤ 5 MB; bounded timeout; no retries beyond $N=3$). Recompute sha256 over the raw bytes; must equal the on-chain patch_hash. Unreachable URL or mismatch = reject. The miner bears hosting responsibility. IPFS is preferred since the CID is the hash.
- **c. Base apply** — git apply --check (no fuzz) against the exact pinned baseline_commit in a fresh checkout. A diff against any other base = reject.
- **d. Surface** — parse the diff's changed-file list (including added files). Every path must match allowed_paths and no path may match denied_paths. New files outside the allowed tree = reject. This runs BEFORE build — it is the primary anti-gaming control: a miner who can touch the harness, tests, scoring, or CI grades their own exam.

- **e. Hermetic build** — apply the patch, build inside the pinned base image with networking disabled (no fetches during build; all deps pre-baked into the image). Build failure = reject with logs persisted to object storage and surfaced to the miner. Enforce a build timeout and resource caps.

Output of Stage 0: a built, content-addressed engine image (tag it with the patch_hash) ready for the correctness gate. Cache it — later stages must run exactly this artifact, never rebuild.

5. Submission state machine

```
committed → fetched → verified → applied → surface_ok → built
                                     ↓
                               (later stages: correctness → screened → benchmarked → scored)
```

```
Any check failure → rejected(reason, evidence_ref) // terminal
Fetch temporarily failing → retry ≤ 3 → rejected(fetch_failed)
```

States are append-only events, not a mutable status column: an audit log of (submission_id, state, timestamp, evidence) is the dispute-resolution substrate. Store gate evidence (fetched bytes hash, apply output, file list, build log) in object storage keyed by patch_hash.

6. Services (v0 deployment shape)

- **chain-watcher** — watches the chain for new commitments on the subnet, creates Submission records, enqueues them. Idempotent on patch_hash.
- **gatekeeper** — workers that pull from the queue and run gates a–d. Cheap, CPU-only, horizontally scalable. Rejections never touch a GPU or a build machine.
- **builder** — runs gate e in isolated containers (network disabled, pinned image, resource-capped). CPU build nodes; no GPU needed at this stage for standard vLLM builds. Emits the built image to the registry tagged by patch_hash.
- **campaign-service** — CRUD for profiles + campaign manifest assembly, pin resolution, consistency validation, and the customer sign-off flow. Freezes the manifest on open.
- **storage** — Postgres for records; object storage for traces, diffs, logs; container registry for base + built images. Queue: anything durable (e.g. Postgres-backed or Redis streams) — volume is low in v0.

All services read the campaign manifest as immutable config. Nothing downstream ever reads the profile directly.

7. Non-goals for v0

- Correctness/quality gate, perf screen, full SLA benchmark, cross-environment validation (Stages 1–4) — designed but not in this build.
- Reward calculation and weight-setting — depends on Stages 1–4.
- Commit-reveal submission scheme — upgrade path noted in §3.
- Multi-campaign scheduling and GPU fleet orchestration — single campaign assumed.

8. Build order (suggested)

- 1. Campaign manifest schema + campaign-service with pin resolution and sign-off. Everything else depends on the manifest.
- 2. Submission schema + chain-watcher (can stub the chain with a local mock that emits commitments).

- 3. Gatekeeper gates a–d as pure functions with unit tests — especially surface-check tests with adversarial diffs (renames into denied paths, added files, symlinks, path traversal in diff headers).
- 4. Builder with hermetic container build + artifact registry.
- 5. End-to-end: mock commitment → built image, with the audit log populated at every transition.

Reference: Inference Optimization Profile (customer-facing input doc) · Owner: Xavier · v0, July 2026